

# Threads on Your Phone

Nothing is downloaded and nothing is sent anywhere. Every number below is measured on the device you are holding. Keep the screen on and the browser in front, or the numbers are meaningless. **Run each section at least twice.** A phone is a real machine but a noisy instrument — we measured the same device 1.3× apart between two sessions, and charging it made things noisier, not steadier. Trust what repeats: where the curve bends, and which worker made things worse. Not the milliseconds.

## 1. What did you bring?

hardwareConcurrency : 8 threads  
deviceMemory : not reported  
platform : MacIntel  
user agent : Mozilla/5.0 (Macintosh; Intel Mac OS X 10\_15\_7)  
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/26.5 Safari/605.1.15

hardwareConcurrency is the number of threads the browser will let you run. Whether that is also the number of cores, and whether those cores are the same as each other, is what the rest of this page is for.

## 2. Does one more thread do one more thread's work?

The same fixed amount of arithmetic, split evenly across 1, 2, 3, ... workers. If threads were free, the bar would double every time you double the workers.

Run scaling sweep

| workers | time   | speedup | efficiency |
|---------|--------|---------|------------|
| 1       | 471 ms | 1.00×   | 100%       |
| 2       | 245 ms | 1.92×   | 96%        |
| 3       | 168 ms | 2.80×   | 93%        |
| 4       | 126 ms | 3.74×   | 93%        |
| 5       | 117 ms | 4.03×   | 81%        |
| 6       | 98 ms  | 4.81×   | 80%        |
| 7       | 86 ms  | 5.48×   | 78%        |

|    |       |       |     |
|----|-------|-------|-----|
| 8  | 82 ms | 5.74x | 72% |
| 9  | 84 ms | 5.61x | 62% |
| 10 | 80 ms | 5.89x | 59% |
| 11 | 79 ms | 5.96x | 54% |
| 12 | 78 ms | 6.04x | 50% |
| 13 | 80 ms | 5.89x | 45% |
| 14 | 78 ms | 6.04x | 43% |
| 15 | 81 ms | 5.81x | 39% |
| 16 | 79 ms | 5.96x | 37% |

Perfect scaling would read 16.00x and 100% on the last row. Write down the worker count where the speedup column stops climbing, and compare it with your 8 reported threads.

### 3. The same loop, handed out two different ways

4096 iterations where iteration  $i$  costs about  $i$  — the last one is thousands of times more expensive than the first. Same total work both times. **Block** cuts the range into contiguous slabs up front. **Dynamic** makes each worker come back for a small chunk whenever it runs out.

Run both schedules

| schedule          | makespan | busiest | idlest | imbalance | vs block |
|-------------------|----------|---------|--------|-----------|----------|
| block (static)    | 558 ms   | 544     | 57     | 1.77      | 1.00x    |
| dynamic, chunk 64 | 377 ms   | 364     | 303    | 1.08      | 1.48x    |
| dynamic, chunk 8  | 336 ms   | 328     | 316    | 1.02      | 1.66x    |

8 workers. Imbalance is the busiest worker divided by the average; 1.00 means everyone worked the same amount. Nobody did less work under dynamic — they just stopped waiting on each other.

### 4. Where the arrows say no

Ten tasks with prerequisites. Adding workers stops helping once the makespan reaches the longest chain, no matter how many workers you have.

[Run task graph](#)

work  $T_1 = 5200$  ms | span  $T_\infty = 2750$  ms | most parallelism worth buying = 1.9 workers

| workers | makespan | lower bound | speedup | efficiency |
|---------|----------|-------------|---------|------------|
| 1       | 5204 ms  | 5200 ms     | 1.00x   | 100%       |
| 2       | 3451 ms  | 2750 ms     | 1.51x   | 75%        |
| 3       | 2752 ms  | 2750 ms     | 1.89x   | 63%        |
| 4       | 2752 ms  | 2750 ms     | 1.89x   | 47%        |
| 6       | 2752 ms  | 2750 ms     | 1.89x   | 31%        |
| 8       | 2752 ms  | 2750 ms     | 1.89x   | 24%        |